iView 实时数据库 REST API 接口文档

1. 用户登录【GET】

URL: /api/userlogin

描述：开启安全管理后需要先用已注册的用户名和密码登录，获取 token，并在之后的请求中携带这个 token。token 有效期为无限期。需要 token 失效时可使用登出 API。未开启安全管理则不需要携带 token。

查询参数

参数	必须	说明
user	是	String，用户名
pass	是	String，密码

示例

请求示例：

GET 方法：http://127.0.0.1:8000/api/userlogin/?user=user1&pass=123456

返回示例：

```
{
    "code": 0,
    "token": "MPzrIAj06iEx9dspMfMgMjdk"
}
```

2. 用户登出【GET】

URL: /api/userlogout

描述：可用此 API 携带 token 进行登出，登出后 token 失效。

查询参数

参数	必须	说明
token	是	String，登录时得到的 token

示例

请求示例：

GET 方法：

http://127.0.0.1:8000/api/userlogout?token=MPzrIAj06iEx9dspMfMgMjdk

返回示例：

登出成功时 code 为 0，code 不为 0 代表失败

```
{
    "code": 0,
    "message": ""
}
```

3. 获取实时数据（点名属性分离）【GET】

URL： /api/realdata

描述：这种方式适合获取多个点的多个属性值时使用。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
tags	是	String，点名不带属性，多个值逗号隔开，如：A1,A2,A3
pars	是	String，属性名，多个值逗号隔开，如：PV,PVTIME,DESC
decimal	否	Integer，返回数值小数点后位数，取值范围 0~9
valueonly	否	Integer，是否只返回值，取值范围 0 或 1，数字 1 代表开启"只返回值"模式，这种模式下，响应体中的数据会十分精简，适合获取的数据量很大，追求极快返回速度的情况。返回的值以数组形式表示，顺序与请求时 tags 参数的顺序相同，详见示例 2

示例 1

请求示例：

GET 方法：

http://127.0.0.1/api/realdata?tags=A1,A2,A3&pars=PV,PVTIME,DESC&token=1234

5678

返回示例：


```
{
    "code": 0,
    "items": [
        {
            "tag": "A1",
            "PV": 100,
            "PVTIME": "2023-04-25T09:28:25.800",
            "DESC": "第一个点"
        },
        {
            "tag": "A2",
            "PV": 98,
            "PVTIME": "2023-04-25T09:28:26.800",
            "DESC": "第二个点"
        },
        {
            "tag": "A3",
            "PV": 50,
            "PVTIME": "2023-04-25T09:28:27.800",
            "DESC": "第三个点"
```

```
        }
    ]
}
```


示例 2（valueonly 模式）

请求示例：

GET 方法：

http://127.0.0.1/api/realdata?tags=A1,A2,A3&pars=PV,DESC&token=12345678&valueonly=1

返回示例：


```
{
    "code": 0,
    "PV": [
        51.0,
        52.0,
        53.0
    ],
    "PVTIME": [
        "2023-04-25T09:28:25.800",
        "2023-04-25T09:28:26.800",
        "2023-04-25T09:28:27.800"
    ],
    "DESC": [
```

"第一个点的描述",

"第二个点的描述",

"第三个点的描述"

]

}

4. 获取实时数据（点名+属性）【GET】

URL: /api/realdata

描述：这种方式适合于获取每个点的某些特定属性值。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
names	是	String，点名带属性，多个值逗号隔开，如：A1.PV, A2.PV, A3.DESC
decimal	否	Integer，返回数值小数点后位数，取值范围 0~9
valueonly	否	Integer，是否只返回值，取值范围 0 或 1，数字 1 代表开启"只返回值"模式，这种模式下，响应体中的数据会十分精简，适合获取的数据量很大，追求极快返回速度的情

		况。返回的值以数组形式表示，顺序与请求时 names 参数的顺序相同，详见示例 2

示例 1

请求示例：

GET 方法：

http://127.0.0.1:8000/api/realdata?names=A1.PV,A1.DESC,A2.PV&token=12345678

返回示例：


```
{
  "code": 0,
  "items": [
    {
      "name": "A1.PV",
      "val": 100.00
    },
    {
      "name": "A1.DESC",
      "val": "第一个点"
    },
    {
      "name": "A2.PV",
```

```
        "val": 50.00
      }
    ]
}
```


示例 2（valueonly 模式）

请求示例：

GET 方法：

http://127.0.0.1:8000/api/realdata?names=A1.PV,A1.DESC,A2.PV&valueonly=1

返回示例：


```
{
  "code": 0,
  "items": [
    100.00,
    "第一个点",
    50.00
  ]
}
```


数组内值的顺序与查询参数 names 的顺序一致，即 A1.PV 对应 100.00，A1.DESC 对应"第一个点"，A2.PV 对应 50.00。这种方式的返回值可读性较差，但体积小，传输效率高，适合对性能有极高要求的场景应对海量实时数据读取的 API，详见 **【获取对外发布数据项某个节点下的所有实时数据】** 或 **【注册项获取实时数据】**。

5. 获取对外发布数据项节点结构【GET】

URL: /api/realdir/{subnode1}/...

描述：REST 服务提供通过对外发布数据项节点名获取该节点下所有点的实时数据的 API，所以会有通过 API 获取对外发布数据项节点结构的需求。

路径参数

参数	说明
subnode1	String，节点名，如：group1

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token

示例 1

请求示例：

GET 方法：http://127.0.0.1:8000/api/realdir

返回示例：

返回树状结构，此例中节点名为 root，其下有两个子节点 group1， group2，group1 下还有一个子节点 level2


```
{
  "items": [
    {
```

```
        "desc": "The Root",
        "items": [
          {
            "desc": "",
            "items": [
              {
                "desc": "",
                "name": "level2"
              }
            ],
            "name": "group1"
          },
          {
            "desc": "",
            "name": "group2"
          }
        ],
        "name": "root"
      }
    ]
  }
```


示例 2

请求示例：

GET 方法：http://127.0.0.1:8000/api/realdir/root/group1

返回示例：


```
{
  "items": [
    {
      "desc": "",
      "name": "level2"
    }
  ]
}
```


6. 获取对外发布数据项某节点下的所有点的实时数据【GET】

URL： /api/realdata/{nodeName1}/{nodeName2}/……

描述：无需除 token 外的查询参数，可直接查询某个节点下的实时数据。响应体内包含【对外发布数据项】内配置的某节点下所有点数据。请求方式类似计算机的文件结构。

注意：要查询的点必须配置在对外发布数据项中

路径参数

参数	说明
nodeName1	String，节点名，如：group1
nodeName2	String，子节点名，如：level2

查询参数

参数	必须	说明
token	否 (开启安全管理后必要)	String，登录时得到的 token
recursion	否	Integer，是否递归取值，0或1，1代表递归，此时当前节点及其所有子节点下面的点值都会被返回，默认为0不递归，详见示例2
valueonly	否	Integer，是否只返回值，取值范围0或1，数字1代表开启"只返回值"模式，这种模式下，响应体中的数据会十分精简，适合获取的数据量很大，追求极快返回速度的情况。返回的值以数组形式表示，顺序与节点中点的顺序相同，详见示例3
decimal	否	Integer，返回数值小数点后位数，取值范围0~9

示例 1

请求示例：

GET 方法：http://127.0.0.1:8000/api/realdata/group1

返回示例：

假设对外发布数据项中配置了【group1】,【group2】两个并列节点。

group1 节点下有一个点值【g1.PV】和一个子节点【level2】, level2 下有一个点值【g2.PV】。

group2 节点下有一个点值【g3.PV】。

则返回的响应体结构如下：

{

"code": 0,

"items": [{

"name": "g1.PV",

"val": 40

}]

}

示例 2（递归）

请求示例：

GET 方法：http://127.0.0.1/api/realdata/group1?recursion=1

返回示例：

递归时，group1 以及它的子节点(level2)下面的所有点都会被返回

{

"code": 0,

"items": [{

"name": "g2.PV",

"val": 50

},

{

"name": "g1.PV",

"val": 40

}

]

}

示例 3（valueonly）

请求示例：

GET 方法：http://127.0.0.1/api/realdata/group1?valueonly=1

返回示例：

这种请求方式适合获取的数据量大，追求极快返回速度的情况。对外发布数据项的【group1】节点下有 A1.PV,A2.PV 两个数据，A1.PV 值为 60，A2.PV 值为 30，则返回如下


```
{
    "code": 0,
    "items": [60,
        30
    ]
}
```


7. 注册点名（用于批量快速取数据）【GET】

URL: /api/realdata

描述：有时我们需要频繁获取一些特定点的数据，而这些点数量较多且分布于不同节点之下。这时我们可以先把这些点在系统中注册成一个特定的名字，之后再通过这个名字简单快速地获取数据。

注意：注册点名必须携带用户登录时得到的 token。每个用户的注册行为是完全独立且相互隔离的。注册完的名字对于其他用户是不可见的。所以即使两个用户恰巧使用同一个名字注册了不同的两组点也不会冲突。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
regname	是	String，注册的名字，相当于给一组点起一个特殊的名字
names	是	String，点名加属性，多个逗号隔开，例如：A1.PV,A2.PV,A3.PV

示例

请求示例：

GET 方法：

http://127.0.0.1:8000/api/realdata?regname=reg01&names=A1.pv,A2.PV,A3.PV&token=12345678

返回示例：

```
{
    "code": 0
}
```

8. 注册项获取实时数据【GET】

URL: api/realdata

描述：将一些点注册成为注册项后，可直接用注册的名字获取这些点的实时数据。获取的实时数据，顺序与注册时的顺序相同。此 API 与注册点名的 API 配合，用于快速获取一组点

的数据。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
regname	是	String，注册项的名称
decimal	否	Integer，返回数值小数点后位数，取值范围 0~9

示例

请求示例：

GET 方法：

http://127.0.0.1:8000/api/realdata?regname=reg01&token=12345678&decimal=2

返回示例：


```
{
    "code": 0,
    "items": [
        10.52,
        8.65,
        89.10
    ]
}
```

9. 设置实时数据【POST】

URL: /api/realdata

描述：利用 POST 方法+请求体的方式设置实时数据。请将需要设置的数据以 JSON 格式写在请求体（request body）中。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token

示例

请求示例：

POST 方法：http://127.0.0.1:8000/api/realdata?token=12345678

请求体：其中根节点名为 items，name 为点名带属性，val 为要设置的值

```
{
    "items": [{
            "name": "A1.PV",
            "val": 40
        },
        {
            "name": "A2.PV",
            "val": 60
        },
        {
```

"name": "A3.DESC",

"val": "第三个点"

}

]

}

返回示例：

code 为 0 表示设置成功，此时 message 为空字符串，code 不为 0 表示失败，此时 message 提示错误信息

{

"code": 1,

"message": "error message xxxxxxxxxxx"

}

10. 设置实时数据（数组方式）【POST】

URL: /api/realdata

描述：利用 POST 方法+请求体的方式设置实时数据。请将需要设置的数据以 JSON 格式写在请求体（request body）中。与上一种方法不同的是

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	string，登录时得到的 token

示例

请求示例：

POST 方法：http://127.0.0.1:8000/api/realdata?token=12345678

请求体：其中 names 为点名带属性的数组，vals 为要设置的值的数组，要求二者顺序一致

```
{
    "names": [
        "A1.PV",
        "A2.PV"
    ],
    "vals": [
        6334,
        26500
    ]
}
```

返回示例：

```
{
    "code": 0
}
```

11. 获取多个时间点历史数据【GET】

URL: /api/hisdata

描述：获取多个时间点历史数据

请求参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
times	是	String，格式：yyyy-MM-DDTHH:mm:ss.zzz（日期与时间用 T 隔开，zzz 毫秒值可不带），多个时间点逗号隔开 如：2022-02-22T14:22:33.444,2022-02-22T15:22:34.555
names	是	点名带属性，多个值逗号隔开，如：A1.PV,A2.DESC
decimal	否	Integer，返回数值小数点后位数，取值范围 0~9
valueonly	否	Integer，是否只返回值，取值范围 0 或 1，数字 1 代表开启"只返回值"模式，这种模式下，响应体中的数据会十分精简，适合获取的数据量很大，追求极快返回速度的情况。返回的值以数组形式表示，按 times 参数排序，详见示例 2

示例 1

请求示例：

GET 方法：

http://127.0.0.1:8000/api/hisdata?times=2022-02-22T14:22:33.444,2022-02-22T15:22:34.555 &names=A1.PV,A2.DESC

返回示例：


```
{
    "code": 0,
    "items": [{
            "name": "A1.PV",
            "vals": [{
                    "time": "2022-02-22T14:22:33.444",
                    "val": 50
                },
                {
                    "time": "2022-02-22T15:22:34.555",
                    "val": 60
                }
            ]
        },
        {
            "name": "A2.DESC",
            "vals": [{
                    "time": "2022-02-22T14:22:33.444",
```

"val": "第二个点"
},
{
"time": "2022-02-22T15:22:34.555",
"val": "第二个点"
}
]
}
]
}

示例 2（valueonly）

请求示例：

GET 方法：

http://127.0.0.1:8000/api/hisdata?times=2022-02-22T14:22:33.444,2022-02-22T15:22:34.555&names=A1.PV,A2.DESC&valueonly=1

返回示例：

{
"code": 0,
"items": [{
"name": "A1.PV",
"vals": [
50,

```
                60
            ]
        },
        {
            "name": "A2.DESC",
            "vals": [
                "第二个点", "第二个点"
            ]
        }
    ]
}
```


12. 获取一段时间内历史数据【GET】

URL: /api/hisdata

描述：获取多个点一段时间内历史数据，注意：除开始时间和结束时间外，还需要设置时间间隔。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
startTime	是	String，格式：yyyy-MM-ddThh:mm:ss.zzz(日期与时间用

		T 隔开，zzz 毫秒可不带)如：2022-01-03T14:25:56.333
endTime	是	String，格式：yyyy-MM-ddThh:mm:ss.zzz(日期与时间用 T 隔开，zzz 毫秒可不带)如：2022-01-03T14:25:58.333
interval	是	Integer,间隔时间毫秒值,2500 代表 2.5 秒
names	是	点名带属性，多个值逗号隔开，如：A1.PV,A2.DESC
valueonly	否	Integer，是否只返回值，取值范围 0 或 1，数字 1 代表开启"只返回值"模式，这种模式下，响应体中的数据会十分精简，适合获取的数据量很大，追求极快返回速度的情况。返回的值以数组形式表示，按时间先后排序，详见示例 2
decimal	否	Integer，返回数值小数点后位数，取值范围 0~9

示例 1

请求示例：

GET 方法：

http://127.0.0.1:8000/api/hisdata?startTime=2022-02-22T15:22:34&endTime=2022-02-22T15:22:37&interval=1000&names=A1.PV,A2.DESC

返回示例：

这种方式在时间点很多时，会返回大量的时间字符串，导致响应体体积过大，响应时间变慢。

当时间点很多，且有性能要求时，应采用示例 2 的方式请求


```
{
    "code": 0,
    "items": [{
            "name": "A1.PV",
            "vals": [{
                "time": "2022-02-22T15:22:34",
                "val": 50
            },
            {
                "time": "2022-02-22T15:22:35",
                "val": 51
            },
            {
                "time": "2022-02-22T15:22:36",
                "val": 52
            }
```

```
            ]
        },
        {
            "name": "A2.DESC",
            "vals": [{
                    "time": "2022-02-22T15:22:34",
                    "val": "第二个点"
                },
                {
                    "time": "2022-02-22T15:22:35",
                    "val": "第二个点"
                },
                {
                    "time": "2022-02-22T15:22:36",
                    "val": "第二个点 xxxx"
                }
            ]
        }
    ]
}
```


示例 2（valueonly）

请求示例：

GET 方法：

http://127.0.0.1:8000/api/hisdata?startTime=2022-02-22T15:22:34&endTime=2022-02-22T15:22:37&interval=1000&names=A1.PV,A2.DESC&valueonly=1

返回示例：


```
{
    "code": 0,
    "items": [{
            "name": "A1.PV",
            "vals": [50, 51, 52]
        },
        {
            "name": "A2.DESC",
            "vals": ["第二个点", "第二个点", "第二个点 xxxx"]
        }
    ]
}
```


数组内值的顺序与时间顺序一致，即 A1.PV 的值 50 对应时间 2022-02-22T15:22:34

由于开始/结束时间及时间间隔已经确定，数组内值对应的时间可计算得到这种方式的返回值体积小，传输效率高，适合对性能有极高要求的场景。

13. 设置历史数据【POST】

URL: /api/hisdata

描述：利用 POST 方法+请求体的方式设置历史数据。请将需要设置的数据以 JSON 格式写在请求体（request body）中。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	string，登录时得到的 token

示例

请求示例：

POST 方法：

http://127.0.0.1/api/hisdata?token=12345678

请求体：


```
{
    "items": [{
            "name": "A1.PV",
            "vals": [{
                "time": "2022-02-22T15:22:34.555",
                "val": 50
            },
            {
                "time": "2022-02-22T15:22:35.555",
```

"val": 51
}
]
},
{
"name": "A2.DESC",
"vals": [{
"time": "2022-02-22T15:22:34.555",
"val": "第二个点"
},
{
"time": "2022-02-22T15:22:35.555",
"val": "第二个点"
}
]
}
]
}

返回示例：

code 为 0 表示设置成功，此时 message 为空，code 不为 0 表示失败，此时 message 提示错误信息

{

"code": 0,

"message": ""

}

14. SQL 查询【POST】

URL: /api/SQL

描述：利用 POST 方法+请求体的方式传入一个带有 SQL 语句的 JSON，并得到类似表结构的返回值。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token

示例

请求示例：

POST 方法：http://127.0.0.1/api/SQL?token=12345678

请求体：

{

"query": "select name, pv, euhi from realdata where kind<250"

}

返回示例：

每个节点相当于表中一列，节点名相当于列名。

{

"code": 0,

"euhi": [
100,
100,
100,
100,
100,
100,
100
],
"name": [
"a1",
"a2",
"a3",
"a4",
"a5",
"a6", "a7"
],
"pv": [-9999, -9999, -9999, -9999, -9999, -9999, -9999]
}

15. 获取报警【GET】

URL: api/alarmdata

描述:带有 startTime 和 endTime 两个查询参数时为获取历史报警,否则为获取实时报警,还可以根据点名（tags），报警优先级（level）等多个参数进行筛选。

查询参数

参数	必须	说明
token	否（开启安全管理后必要）	String，登录时得到的 token
realalmcount	否	Integer，最多取得的条数
org	否	String，指定要查询的点所在目录
unit	否	Integer，指定要查询的点所在单元
level	否	Integer，报警优先级，获取不小于该级别的报警
tags	否	String，点名列表，多个点逗号隔开，如 A1,A2,A3
groupbytag	否	Integer，是否按点名分组，为 1 时分组，为 0 时不分组
startTime	否（携带后表示取历史报警）	String，格式：yyyy-MM-ddThh:mm:ss.zzz(日期与时间用 T 隔开，zzz 毫秒可不带) 如：2022-01-03T14:25:56.333

endTime	否（携带后表示取历史报警）	String，格式：yyyy-MM-ddThh:mm:ss.zzz(日期与时间用T隔开，zzz 毫秒可不带)如：2022-01-03T14:25:56.333

示例

请求示例：

GET 方法：

http://127.0.0.1/api/alarmdata?token=12345678

返回示例：

各字段说明在返回值示例下方


```
{
    "code": 0,
    "items": [{
        "ack": 2,
        "cat": 38,
        "desc": "",
        "eu": "",
        "group": "root.groupB",
        "level": 1,
        "limit": 80,
        "name": "B2",
        "oper": "",
```

"parno": 38,
"prio": 1,
"time": "2022-03-23T13:32:37",
"unit": 0,
"val": 89.400000000000006
}, {
"ack": 2,
"cat": 36,
"desc": "",
"eu": "",
"group": "root.groupA",
"level": 2,
"limit": 10,
"name": "A1",
"oper": "",
"parno": 36,
"prio": 2,
"time": "2022-03-23T13:32:37",
"unit": 0,
"val": 3.5
}]
}

ack 确认状态枚举值，0 未确认，1 确认，2 恢复，3 禁止（禁止说明该点的报警被禁止掉了，但会保存一条记录）；

cat 类别，对应报警配置中的 ID，模拟点默认情况下，36 低低报，37 低报，38 高报，39 高高报，多数情况下使用默认配置即可；

desc 说明；

eu 单位；

group 层级，本例中点 B2 在 groupB 节点下，所以字段值为"root.groupB" level 报警级别枚举值，0 低级，1 高级，2 紧急，在点的报警界面配置；

limit 限值，分为低低限，下限，上限，高高限，在点的报警界面配置。本例中分别为 10，20，80，90 name 点名，如 A1，A2，B2；

oper 确认者；

parno 类型 ID，模拟点默认情况下此字段与 cat 字段相同 prio 优先级，对于模拟点和数字点，此字段等同于 level time 报警时间；

unit 点所在的单元，默认为 0，可在点设置中配置；

val 点值；